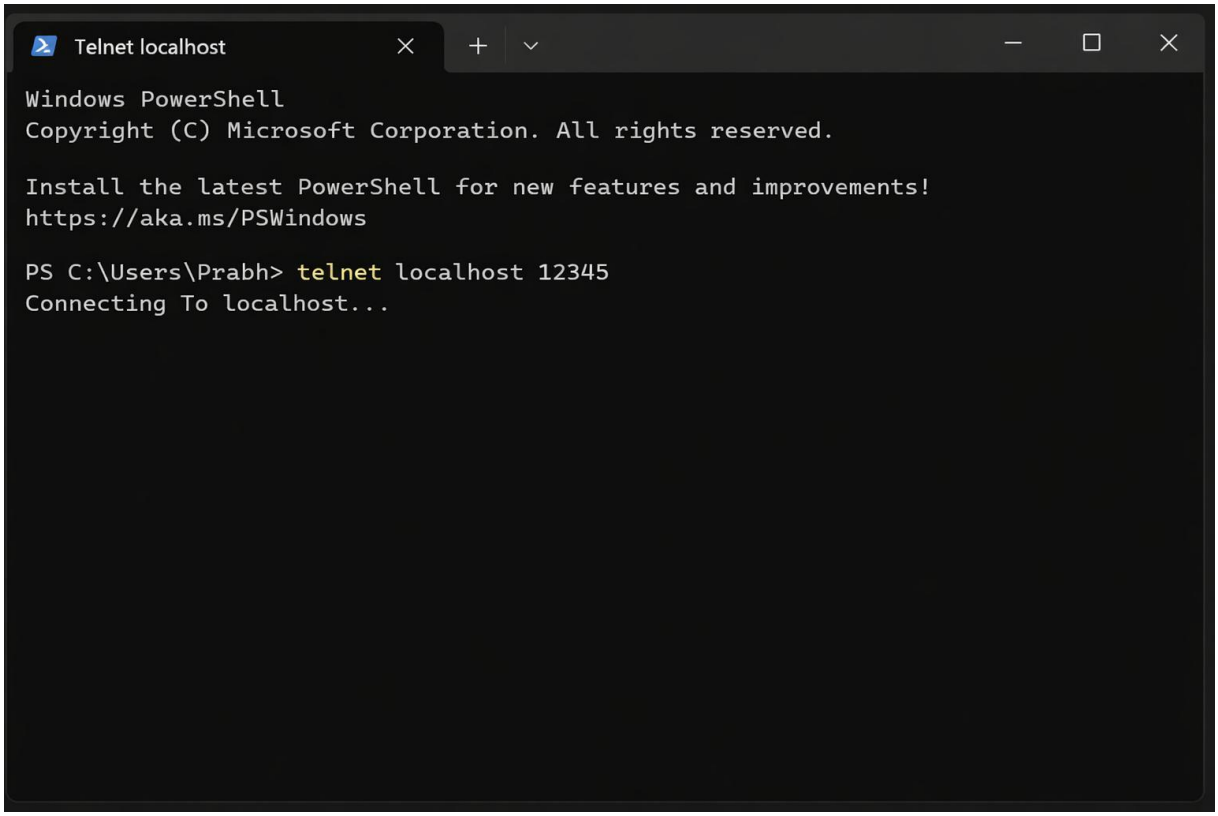
Assesment-4

Name:N.P.SANJAY

REG.NO: 23BCE0211

1.Develop a simple client server model using telnet and capture the packets transmitted with tshark. Analyze the pcap file and get the transmitted data (plain text) using any packet capturing library.
Implement the above scenario using SSH and observe the data



```
Telnet localhost

 Message receivedeMessage receivedlMessage receivedlMessage receivedoMessage received Message receivedwMessage receivedo
Message receivedrMessage receivedlMessage receivedd Message received Message received
```

```
Telnet localhost

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvementshttps://PSWindows

PS C:\Users\Prabh> tshark -D
 1.  Device\NPF_{A1BF45D4-95514391-EFF809F9900} (Local Area Connection* 10)
 2.  Device\NPF_{1DBAF136-6AAF-88A9-B447268276} (Local Area Connection* 9)
 3.  Device\NPF_{DPFC322E-1DF3-8842-5A5F88C32P} (Local Area Connection* 8)
 4.  Device\NPF_{DFC8892E-1DF3-0842-5ADFA80CCD} (Bluetooth Network Connection)
 5.  Device\NPF_{E097E2AC-0700-B550-5BEC29BD33} (Wi-Fi)
 6.  Device\NPF_{407A716F-0760-9885-55BB28BDCC} (Local Area Connection* 2)
 7.  Device\NPF_{09667098-6A80-BBA1-12D084MEF2} (Local Area Connection* 1)
 8.  Device\NPF_{09670C67-0CB2-4TT1-87D965EFA1} (Ethernet)
 9.  Device\NPF_{E38E1573-A43C-47F1-902B-C2409221E879)
 10. etwdump (Event Tracing for Windows (ETW) reader)
 11. Device\NPF_Loopback (Adapter for loopback.pcap)

PS C:\Users\Prabh> tshark -i 9 -w capture.pcap
Capturing on 'Adapter for loopback traffic capture'
635

PS C:\Users\Prabh>
```

```
PS C:\Users\Prabh> tshark -i 9 -w capture.pcap
Capturing on 'Adapter for loopback traffic capture'
635
```

```python
import socket

server = socket.socket()
server.bind(('localhost', 12345))
server.listen(1)

print("Server waiting...")

conn, addr = server.accept()
print("Connected from", addr)

while True:
    data = conn.recv(1024)
    if not data:
        break
    print("Client:", data.decode())
    conn.send(b"Message received")

conn.close()
```

```
Server waiting...
Connected from ('127.0.0.1', 50199)
Client: h
Client: e
Client: l
Client: l
Client: o
Client:
Client: w
Client: o
Client: r
Client: l
Client: d
Client:
```

This decodes to Hello World

```
PS C:\Users\Prabh> tshark -r capture.pcap -Y "tcp.port==12345" -T fields -e data
68
4d657373661676520726563656669766564
69
74
4d657373661676520726563656669766564
69
74
4d657373661676520726563656669766564
69
74
4d657373661676520726563656669766564
69
74
4d657373661676520726563656669766564
69
73
4d657373661676520726563656669766564
69
73
4d657373661676520726563656669766564
69
4d657373616765207265563656569766564
```

```
4d657373616765207265636569766564

73

4d657373616765207265636569766564

69

4d657373616765207265636569766564

73

4d657373616765207265636569766564

6d

4d657373616765207265636569766564

65

4d657373616765207265636569766564
```

2. Develop a web application that implements JSON web token

Title :

Implementation of JWT-based Authentication using Flask

Aim :

To develop a web application that uses JSON Web Token (JWT) for secure authentication.

Tools Used :

- Python
- Flask
- PyJWT
- Windows Terminal

Procedure :

Step 1: Create Project Folder

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\prabh> mkdir assessment4_q2


    Directory: C:\Users\prabh


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        30-03-2026     18:36                assessment4_q2


PS C:\Users\prabh> cd assessment4_q2
```

Step 2: Create Flask Application

- Created file app.py
- Implemented login and protected routes using JWT

app.py:

```python
from flask import Flask, request, jsonify
import jwt
import datetime

app = Flask(__name__)
SECRET_KEY = "mysecretkey"

@app.route('/login', methods=['POST'])
def login():
    data = request.json
    if data['username'] == "admin" and data['password'] == "1234":
        token = jwt.encode({
            'user': data['username'],
            'exp': datetime.datetime.utcnow() + datetime.timedelta(minutes=30)
        }, SECRET_KEY, algorithm="HS256")
        return jsonify({'token': token})
    return jsonify({'message': 'Invalid credentials'}), 401

@app.route('/protected', methods=['GET'])
def protected():
    token = request.headers.get('Authorization')
    if not token:
        return jsonify({'message': 'Token missing'}), 403
    try:
        jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
        return jsonify({'message': 'Access granted'})
    except:
        return jsonify({'message': 'Invalid token'}), 403

app.run()
```

Step 3: Install Required Packages

```
PS C:\Users\prabh\assessment4_q2> py -m pip install flask pyjwt
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting pyjwt
  Downloading pyjwt-2.12.1-py3-none-any.whl.metadata (4.1 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp312-cp312-win_amd64.whl.metadata (2.8 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.7-py3-none-any.whl.metadata (4.0 kB)
Collecting colorama (from click>=8.1.3->flask)
  Downloading colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading pyjwt-2.12.1-py3-none-any.whl (29 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.3.1-py3-none-any.whl (108 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading markupsafe-3.0.3-cp312-cp312-win_amd64.whl (15 kB)
Downloading werkzeug-3.1.7-py3-none-any.whl (226 kB)
```

Step 4: Run the Application

py app.py

```
Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Installing collected packages: pyjwt, markupsafe, itsdangerous, colorama, blinker, werkzeug, jinja2, click, fl
ask
  WARNING: The script flask.exe is installed in 'C:\Users\prabh\AppData\Local\Programs\Python\Python312\Script
s' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-loca
tion.
Successfully installed blinker-1.9.0 click-8.3.1 colorama-0.4.6 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 ma
rkupsafe-3.0.3 pyjwt-2.12.1 werkzeug-3.1.7

[notice] A new release of pip is available: 24.2 -> 26.0.1
[notice] To update, run: C:\Users\prabh\AppData\Local\Programs\Python\Python312\python.exe -m pip install --up
grade pip
PS C:\Users\prabh\assessment4_q2> py app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server
instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Output:**Running on http://127.0.0.1:5000/**

Step 5: Generate JWT Token(In NEW Terminal)

Command:

Invoke-RestMethod -Uri http://127.0.0.1:5000/login -Method POST -Headers @{"Content-Type"="application/json"} -Body '{"username":"admin","password":"1234"}'

Output:

```
PS C:\Users\prabh> (Invoke-RestMethod -Uri http://127.0.0.1:5000/login -Method POST -Headers @{"Content-Type"="applicati
on/json"} -Body '{"username":"admin","password":"1234"}').token
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiYWRtaW4iLCJleHAiOjE3NzQ4Nzg4NTl9.CuN4f4vy0N53abax7NgLQSSVjxIoHTAjWZAXAX
4jgO0
```

Step 6: Access Protected Route

Command:

Invoke-RestMethod -Uri http://127.0.0.1:5000/protected -Method GET -Headers @{"Authorization"="<JWT_TOKEN>"}

Output: Access granted

```
PS C:\Users\prabh> Invoke-RestMethod -Uri http://127.0.0.1:5000/protected -Method GET -Headers @{"Authorization"="eyJhbG
ciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiYWRtaW4iLCJleHAiOjE3NzQ4Nzg4NTl9.CuN4f4vy0N53abax7NgLQSSVjxIoHTAjWZAXAX4jgO0"
}

message
-------
Access granted
```

Server Execution Logs Showing JWT Authentication Requests and Responses

```
C:\Users\prabh\assessment4_q2\app.py:14: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UT
C).
  'exp': datetime.datetime.utcnow() + datetime.timedelta(minutes=30)
C:\Users\prabh\AppData\Local\Programs\Python\Python312\Lib\site-packages\jwt\api_jwt.py:147: InsecureKeyLengthWarning: T
he HMAC key is 11 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section
 3.2.
  return self._jws.encode(
127.0.0.1 - - [30/Mar/2026 18:50:13] "POST /login HTTP/1.1" 200 -
C:\Users\prabh\AppData\Local\Programs\Python\Python312\Lib\site-packages\jwt\api_jwt.py:365: InsecureKeyLengthWarning: T
he HMAC key is 11 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section
 3.2.
  decoded = self.decode_complete(
127.0.0.1 - - [30/Mar/2026 18:51:58] "GET /protected HTTP/1.1" 403 -
127.0.0.1 - - [30/Mar/2026 18:52:13] "GET /protected HTTP/1.1" 403 -
127.0.0.1 - - [30/Mar/2026 18:53:25] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [30/Mar/2026 18:54:19] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [30/Mar/2026 18:55:11] "GET /protected HTTP/1.1" 200 -
```